

Detailed KAI Usage Report

Project: modern_notification

Goal: create a Delphi/VCL component that displays Windows 10/11 style notifications with embedded PNG icons.

Project path: C:\Users\<user>\Documents\Embarcadero\Studio\Projets\modern_notification

RAD Studio project: modern_notification.dproj

Package: modern_notification.dpk

Final state checked: Win32 / Release , full build **OK**.

1. Introducing Embarcadero KAI

KAI is Embarcadero's IDE-aware assistant for RAD Studio. It works directly with the active Delphi environment: the open project, editor buffers, unsaved changes, project files, design-time packages and compiler output.

In this project, KAI was used as a development assistant to transform a simple notification procedure into an installable Delphi component with embedded resources, design-time property editors, palette icons and a written implementation report.

Main Strengths

- **IDE-first workflow:** KAI reads RAD Studio editor buffers before reading files directly from disk.
- **Protection of unsaved changes:** edits are applied to the IDE buffer when the file is open.
- **Active project awareness:** KAI can inspect the package, platform, configuration and project file list.
- **Targeted source edits:** changes can be applied precisely with `applyEdit` , without rewriting unrelated code.
- **Project management:** KAI can add units and resource files to the active RAD Studio project.
- **Continuous validation:** KAI can compile the project through RAD Studio after each important change.
- **Context-aware Delphi support:** the implementation was adapted to VCL, design-time packages, BPL resources and WinRT toast APIs.
- **Traceability:** the workflow can be documented as a reproducible report.

KAI Operations Used

KAI tool	Use in this project
<code>getProjectInfo</code>	Identify the active project, platform, configuration, framework type and package files.
<code>listOpenFiles</code>	Check which files were open in RAD Studio before reading or editing them.
<code>getEditorContent</code> / <code>getEditorLines</code>	Read the actual IDE buffer contents, including unsaved changes.
<code>getModificationStatus</code>	Check whether a file was open and modified before applying changes.
<code>setEditorContent</code>	Replace an entire open file when a full controlled rewrite was appropriate.
<code>applyEdit</code>	Apply targeted changes inside open IDE buffers such as <code>notify_Unit.pas</code> and <code>modern_notification.dpk</code> .
<code>addFileToProject</code>	Add new units and resource files to the RAD Studio project.
<code>compileProjects</code>	Build the package repeatedly to verify each development step.

Source Modification Tools

For source files open in RAD Studio, modifications were made with KAI IDE tools, mainly `applyEdit` for targeted changes and `setEditorContent` for complete controlled replacements. This keeps the IDE editor buffer as the source of truth and avoids losing unsaved work.

PowerShell was also used, but as a complementary system tool rather than the primary editor for open RAD Studio source files.

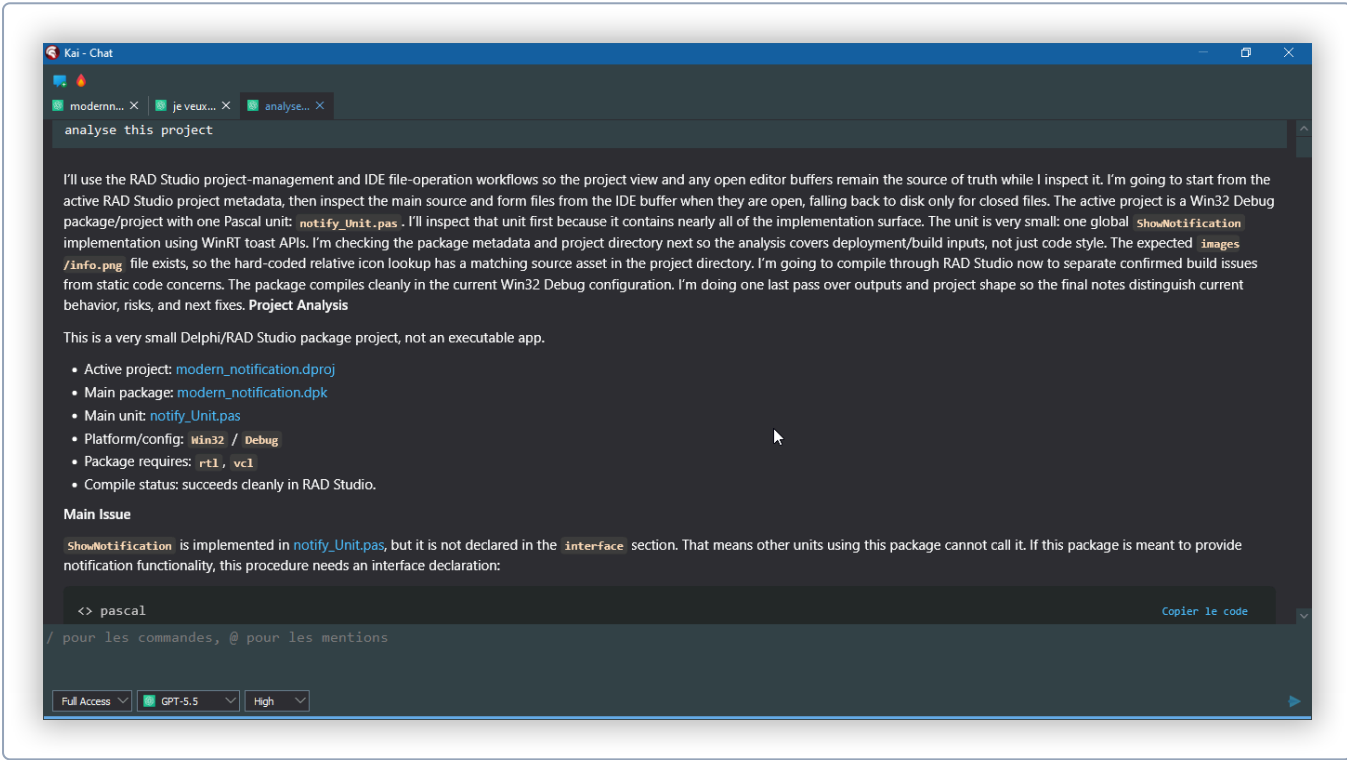
Tool	Exact role in this project
KAI <code>applyEdit</code>	Modify a precise part of an open RAD Studio file, for example <code>notify_Unit.pas</code> or <code>modern_notification.dpk</code> .
KAI <code>setEditorContent</code>	Replace the full contents of an open file when that was simpler and controlled, especially for small resource files.
KAI <code>addFileToProject</code>	Register a new unit or resource file inside the RAD Studio project.

Tool	Exact role in this project
PowerShell	List files, check PNG dimensions, generate intermediate bitmaps, verify BPL resources, redact screenshots and convert HTML to PDF.

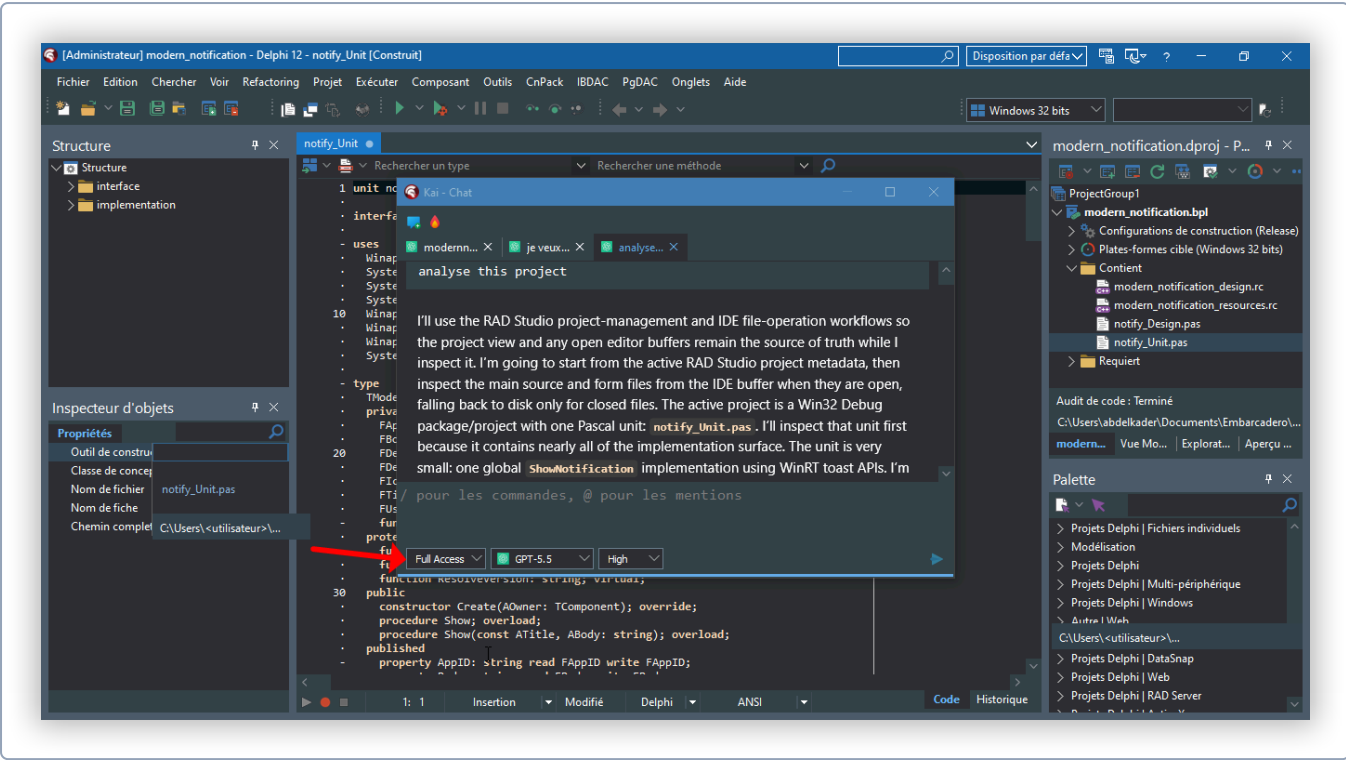
Applied principle: when a file was open in RAD Studio, KAI was used as the source of truth. Direct disk access was reserved for closed files, resource checks and report generation.

2. Screenshots

The following screenshots come from the `shots` folder. They document the use of KAI during the analysis and development of the component inside RAD Studio.



Screenshot 1 - Initial project analysis in the KAI chat window.



Screenshot 2 - KAI used alongside RAD Studio with the modern_notification package open.

3. Developer-KAI Discussion

This section summarizes the functional discussion that guided the implementation. It keeps the developer's requests, KAI's decisions and the validation steps.

Step	Developer request	KAI response / action
1	Analyze the existing project.	KAI inspected the active project metadata, open files and main unit. The project was identified as a Delphi package containing an initial toast notification procedure.
2	Create a Delphi component that displays a Windows 10/11 style notification with a large icon.	KAI created <code>TModernNotification</code> , published properties, the <code>Show</code> method, the global <code>ShowNotification</code> helper and design-time registration.
3	Embed <code>info.png</code> inside the BPL.	KAI added an <code>RCDATA</code> resource and implemented temporary extraction from the BPL before displaying the toast.
4	Allow the developer to use a 64x64 icon; otherwise use the embedded icon.	The component now gives priority to <code>IconFileName</code> when the file exists, then falls back to the embedded icon.

Step	Developer request	KAI response / action
5	Add <code>notification.png</code> as the small component icon.	KAI converted the image into a Delphi-compatible bitmap resource and declared <code>TMODERNNOTIFICATION</code> for the design-time palette icon.
6	Add a <code>DefaultIcon</code> true/false property.	KAI added a published Boolean property, defaulting to <code>True</code> . If set to <code>False</code> , no embedded fallback icon is used.
7	Use a file dialog for the icon path, like <code>TImage</code> .	KAI created <code>notify_Design.pas</code> and a property editor for <code>IconFileName</code> with a <code>...</code> button and an image-filtered <code>TOpenDialog</code> .
8	Fix the default icon and the small component icon because they were not displayed.	KAI corrected resource loading with <code>FindClassHInstance(TModernNotification)</code> and linked the design-time resource in <code>notify_Design.pas</code> .
9	Explain the purpose of <code>UseLargeIcon</code> .	KAI clarified that it forces the image to be used as the large toast icon through <code>placement="appLogoOverride"</code> and <code>hint-crop="none"</code> .
10	Embed more 64x64 PNG images from the <code>images</code> folder.	KAI added an embedded icon table, the <code>DefaultIconName</code> property and a design-time drop-down list.
11	Add the new 64x64 images from the <code>images</code> folder.	KAI synchronized <code>modern_notification_resources.rc</code> and the <code>EmbeddedIcons</code> table. The BPL was verified with 27 <code>MODERN_NOTIFICATION_ICON_*</code> resources.
12	Clean unused functions/code.	KAI checked references, kept functions used by properties or design-time code, and removed unused units: <code>Winapi.WinRT</code> and <code>System.TypInfo</code> .
13	Create a detailed PDF report of the KAI workflow.	KAI generated an HTML report and converted it to PDF with Microsoft Edge headless after checking a final package build.
14	Translate the report into English without overwriting the French report.	KAI created this separate English HTML/PDF report using <code>_en</code> filenames, preserving the original French report.

Synthetic Conversation Extract

```
Developer: analyze this project
KAI: Inspect the active project, open files and baseline build status.

Developer: the goal is to create a Delphi component that displays a notification.
KAI: Create TModernNotification, published properties and Show methods.

Developer: I want info.png embedded in the BPL.
KAI: Add an RCDATA resource and extract it from the BPL.

Developer: the developer can use a 64x64 icon, otherwise use the embedded icon.
KAI: Prioritize IconFileName and fall back to the embedded icon.

Developer: add DefaultIcon true/false.
KAI: Add the DefaultIcon property and no-icon behavior.

Developer: why not use a dialog like TImage?
KAI: Add a design-time property editor with a "..." button.

Developer: the default icon and small component icon are not displayed.
KAI: Fix module handle resource loading and palette icon resource linking.

Developer: embed more 64x64 PNG images.
KAI: Add DefaultIconName, a drop-down list and multiple embedded resources.

Developer: clean the code.
KAI: Remove unused uses and rebuild.

Developer: create a PDF report.
KAI: Generate the report and convert it to PDF.
```

4. Component Creation Steps

The following sections describe the technical implementation steps: project analysis, component creation, Windows toast handling, embedded resources, design-time support, validation and maintenance.

4.1 Initial Project Analysis

KAI first inspected the existing package. The project contained mainly `notify_Unit.pas`, `modern_notification.dpk`, `modern_notification.dproj` and related resource files.

The initial unit contained a global `ShowNotification` procedure using WinRT toast APIs: `TToastNotificationManager`, `IToastNotification` and Windows toast XML templates. It was not yet exposed as an installable component.

KAI compiled the project to establish a valid starting point. The baseline build succeeded, which allowed the component to be expanded step by step with validation after each important change.

4.2 Transformation into a Delphi Component

The runtime code was reorganized around a non-visual component:

```

type
  TModernNotification = class(TComponent)
  published
    property AppID: string;
    property Body: string;
    property DefaultIcon: Boolean;
    property DefaultIconName: string;
    property IconFileName: string;
    property Title: string;
    property UseLargeIcon: Boolean;
    property Version: string;
  public
    procedure Show; overload;
    procedure Show(const ATitle, ABody: string); overload;
  end;

```

The global `ShowNotification` helper was kept so that a notification can still be displayed without dropping the component on a form.

4.3 Windows Notification Display

The component builds a toast XML document from Windows templates:

- `ToastImageAndText02` when an icon is available.
- `ToastText02` when no icon should be displayed.

Text values are written into the XML `text` nodes. When an image is used, it is converted to a local `file:///...` URI and assigned to the XML `image` node.

The `UseLargeIcon` property adds:

```

placement="appLogoOverride"
hint-crop="none"

```

This produces a large Windows 10/11 style notification icon.

4.4 Default Icon Embedded in the BPL

The default icon was embedded through `modern_notification_resources.rc`, then compiled into `modern_notification_resources.res`.

Because toast notifications need a file URI for local images, the embedded PNG is extracted to the Windows temporary folder before being used.

```

LModuleHandle := FindClassHInstance(TModernNotification);
LResourceStream := TResourceStream.Create(LModuleHandle, ResourceName, RT_RCDATA);

```

Using `FindClassHInstance` ensures that the resource is loaded from the BPL containing the component.

4.5 External Icon or Embedded Icon

The final resolution order is:

1. If `IconFileName` points to an existing file, that file is used.
2. If `DefaultIcon=False`, no icon is used.
3. Otherwise the resource selected by `DefaultIconName` is extracted and used.
4. If the name is empty or invalid, the component falls back to `button-info`.

4.6 Embedded 64x64 PNG Images

The 64x64 PNG files from the `images` folder were declared as `RCDATA` resources. The component maintains an internal table linking the visible icon name, the Windows resource name and the extracted temporary file name.

Example values exposed by `DefaultIconName`:

```
button-cancel
button-download
button-error
button-help
button-info
button-ok
button-refresh
button-trend-down
button-update
chart-pie-2d
chart-pie-2d-exploded
charts
charts-colors
emoticon-happy
emoticon-sad
emoticon-smile
find
fire
gift
internet
light-bulb
new
on-off
reminder
save
user-profile
```

4.7 Design-Time Unit

A separate `notify_Design.pas` unit was created to isolate code that depends on `designide`, `DesignIntf`, `DesignEditors` and `ToolsAPI`.

It registers the component:


```
RegisterComponents('Modern Notification', [TModernNotification]);
```

It also registers property editors:

- `IconFileName`: ... button opening a `TOpenDialog`.
- `DefaultIconName`: drop-down list of embedded icons.
- `Version`: read-only property.

4.8 Component Icon and RAD Studio Splash

The design-time resource file contains:

```
TMODERNNOTIFICATION BITMAP "images\\flag-green.bmp"  
TMODERNNOTIFICATION_32 BITMAP "images\\flag-green_32.bmp"
```

`TMODERNNOTIFICATION` is the resource name RAD Studio expects for the component palette icon.

`TMODERNNOTIFICATION_32` is used for the splash/plugin bitmap.

```
{$R 'modern_notification_design.res'}
```

4.9 Code Cleanup

A cleanup pass was performed after the main additions. Functions still present are used directly or indirectly by published properties, runtime behavior or design-time initialization.

Unused units removed:

- `Winapi.WinRT` from `notify_Unit.pas`.
- `System.TypeInfo` from `notify_Design.pas`.

4.10 Validation

Project	<code>modern_notification.dproj</code>
Framework	<code>VCL</code>
Platform	<code>Win32</code>
Configuration	<code>Release</code>
Build	OK, no messages

4.11 Final Component Usage

Using an embedded icon:

```
ModernNotification1.Title := 'Information';
ModernNotification1.Body := 'Processing completed';
ModernNotification1.DefaultIcon := True;
ModernNotification1.DefaultIconName := 'button-info';
ModernNotification1.Show;
```

Using an external icon:

```
ModernNotification1.Title := 'Alert';
ModernNotification1.Body := 'Operation completed with an error';
ModernNotification1.IconFileName := 'C:\Images\error_64.png';
ModernNotification1.Show;
```

Quick use without dropping the component on a form:

```
ShowNotification('Information', 'Processing completed');
```

4.12 Main Files Created or Modified

File	Role
<code>notify_Unit.pas</code>	Runtime component code, toast display, embedded icon extraction and published properties.
<code>notify_Design.pas</code>	Design-time registration, property editors and design-time icon/splash resources.
<code>modern_notification_resources.rc</code>	64x64 PNG declarations as <code>RCDATA</code> resources.
<code>modern_notification_design.rc</code>	Component icon bitmap and splash/plugin bitmap.
<code>modern_notification.dpk</code>	Delphi package containing runtime/design-time units and the <code>designide</code> dependency.
<code>modern_notification.dproj</code>	RAD Studio project with unit and resource references.

4.13 Maintenance Notes

- After changing a design-time resource, restart RAD Studio or reinstall the package to see the new palette icon.
- Windows toast notifications use file URIs, so embedded PNG resources are extracted before display.
- To add another embedded icon, add a 64x64 PNG to `images`, declare it in the `.rc` file and add it to `EmbeddedIcons`.

- Continue using KAI for files open in the IDE to avoid overwriting unsaved changes.